

ANTHROPIC · ENGINEERING

효과적인 AI 에이전트 구축하기

복잡한 프레임워크가 아닌,
단순하고 결합 가능한 패턴부터 시작하라

Building Effective Agents

Anthropic Engineering · anthropic.com/engineering



KEY MESSAGE

핵심 메시지



성공은 가장 **정교한 시스템**을 만드는 것이 아니라,
필요에 맞는 **올바른 시스템**을 만드는 것이다.

— Anthropic, Building Effective Agents

WHAT ARE AGENTS?

에이전트란 무엇인가

두 가지 카테고리의 구분

01 · WORKFLOW

워크플로우

LLM과 도구가 **사전 정의된 코드 경로**를 통해 조율되는 시스템.

- ▶ 경로·순서가 개발자에 의해 결정됨
- ▶ 예측 가능성과 제어가 중요할 때
- ▶ 5가지 패턴으로 조합 (체이닝·라우팅·병렬화·오케스트레이터·평가자)

02 · AGENT

에이전트

LLM이 자신의 **프로세스와 도구 사용을 동적으로 지휘**하는 시스템.

- ▶ 환경 피드백을 보고 다음 행동을 스스로 결정
- ▶ 유연성과 모델 판단이 중요할 때
- ▶ 비용·오류 누적 대비 가드레일 필수

DECISION GUIDE

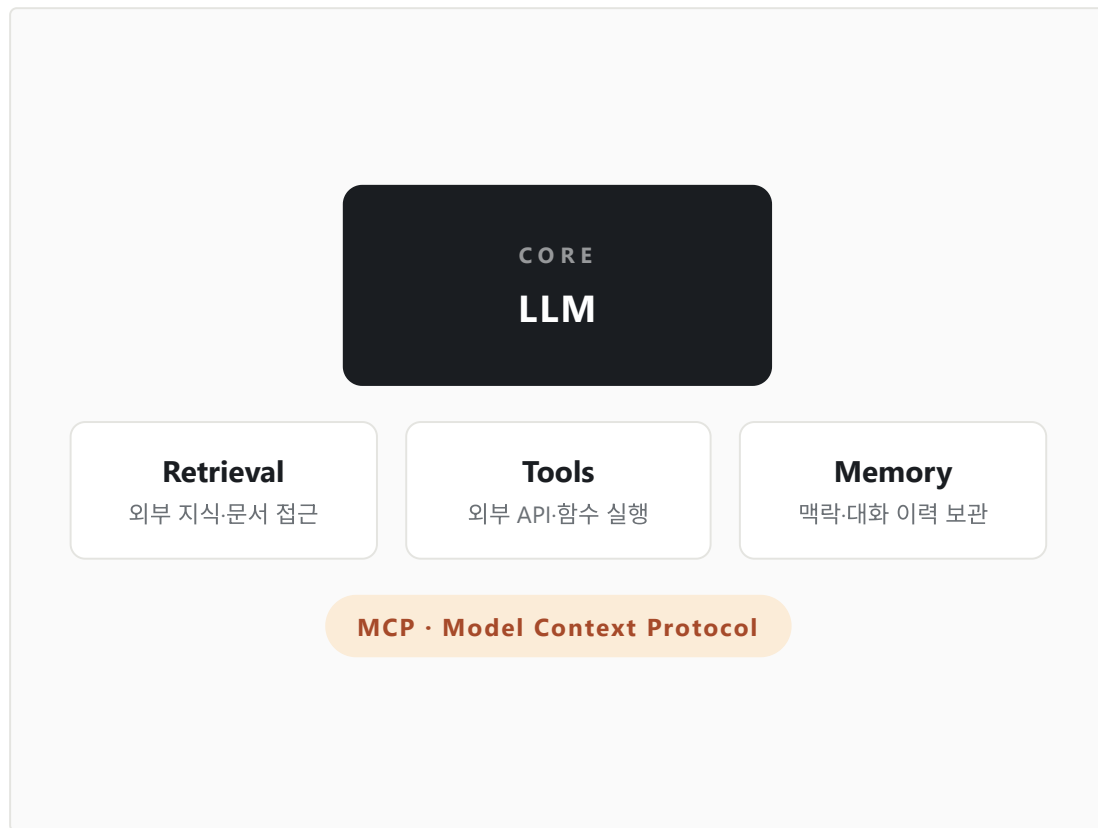
언제 에이전트를 사용할까

사용해야 할 때	사용하지 말아야 할 때
✓ 개방형 문제 · 작업을 미리 예측할 수 없음	✗ 단일 호출 + 검색 · 예시로 충분한 경우
✓ 유연성과 모델 기반 의사결정이 필요	✗ 레이턴시와 비용이 결정적으로 중요
✓ 신뢰할 수 있는 환경에서 확장이 필요	✗ 검증된 고정 경로가 이미 존재

권장 출발점 — 단순한 프롬프트에서 시작하고, 종합적 평가로 최적화하라. 더 단순한 솔루션이 부족할 때에만 복잡성을 추가하라.

FOUNDATION

기본 빌딩 블록 · Augmented LLM



FOUNDATION

모든 에이전트 시스템의 출발점

검색(Retrieval), 도구(Tools), 메모리(Memory)로 강화된 LLM이 모든 에이전트 시스템의 기반입니다.

MCP(Model Context Protocol) 는 이들을 연결하는 표준 인터페이스입니다. 한 번의 통합으로 여러 도구·데이터 소스에 접근할 수 있습니다.

WORKFLOWS

5가지 워크플로우 패턴

Five composable building patterns

01 프롬프트 체이닝 — 작업을 순차적 단계로 분해 (Prompt Chaining)

SEQUENCE

02 라우팅 — 입력을 분류해 전문 프로세스로 안내 (Routing)

CLASSIFY

03 병렬화 — 동시에 수행하고 결과를 통합 (Parallelization)

PARALLEL

04 오케스트레이터-워커 — 중앙 LLM이 동적 배분 (Orchestrator)

DELEGATE

05 평가자-최적화 — 피드백 루프로 완성도 상승 (Evaluator)

LOOP

PATTERN 01

프롬프트 체이닝

WORKFLOW 01

순차적 단계로 분해

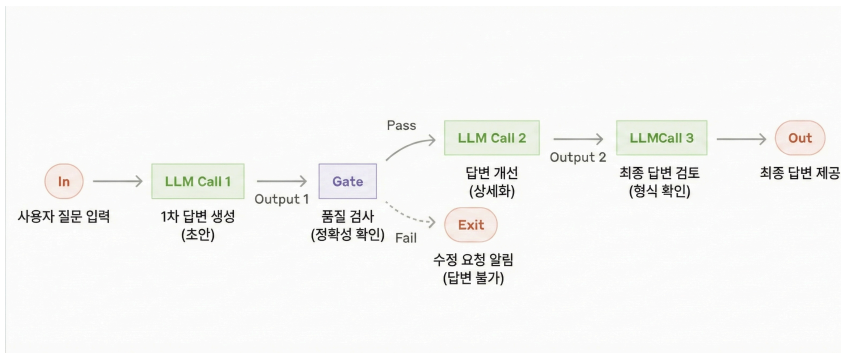
PROMPT CHAINING

하나의 큰 작업을 도미노처럼 여러 단계로 쪼개고, 이전 LLM의 출력이 다음 LLM의 입력이 됩니다. 각 단계가 단순해지므로 정확도가 비약적으로 상승합니다.

비유 이어달리기에서 baton을 전달하는 계주 선수들

조건 작업이 고정된 하위 단계로 명확히 분해될 때

예시 마케팅 문구 작성 → 다른 언어로 번역



PATTERN 02

라우팅

WORKFLOW 02

분류해서 적재적소로

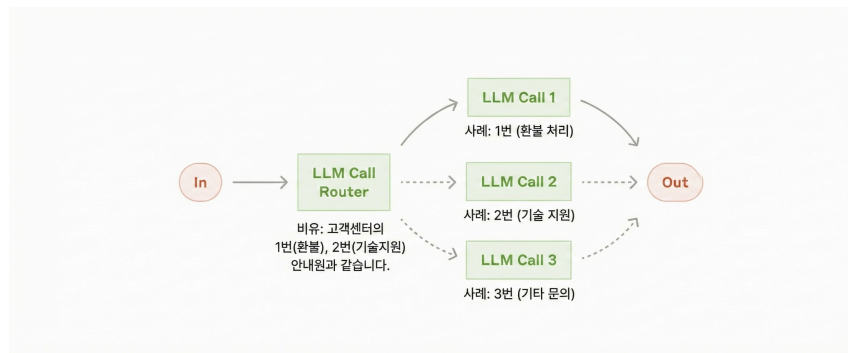
ROUTING

분류 모델이 입력을 판단해 **가장 적합한 전문 프로세스**로 안내합니다. 모든 케이스를 한 프롬프트에 옥여넣을 때 발생하는 성능 저하를 방지합니다.

비유 고객센터의 1번(환불), 2번(기술지원) ARS 안내

조건 명확히 구분되는 카테고리가 존재할 때

예시 간단한 질문은 Haiku, 어려운 질문은 Sonnet으로



PATTERN 03

병렬화

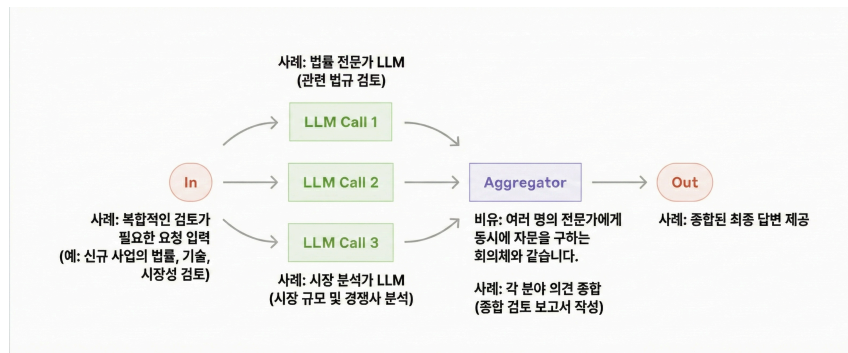
WORKFLOW 03

동시에 수행, 결과를 통합

PARALLELIZATION

여러 LLM이 **동시에 업무**를 수행하고 결과를 하나로 모읍니다. 독립 하위 작업 병렬 실행(Sectioning)과 동일 작업 다중 실행 후 투표(Voting) 두 변형이 있습니다.

비유	여러 전문가에게 동시에 자문을 구하는 회의체
장점	속도 + 여러 관점을 반영한 결과 신뢰도 상승
예시	코드 보안 취약점을 3개 모델이 각기 다른 관점으로 검토



PATTERN 04

오케스트레이터-워커

WORKFLOW 04

동적으로 배분하고 종합

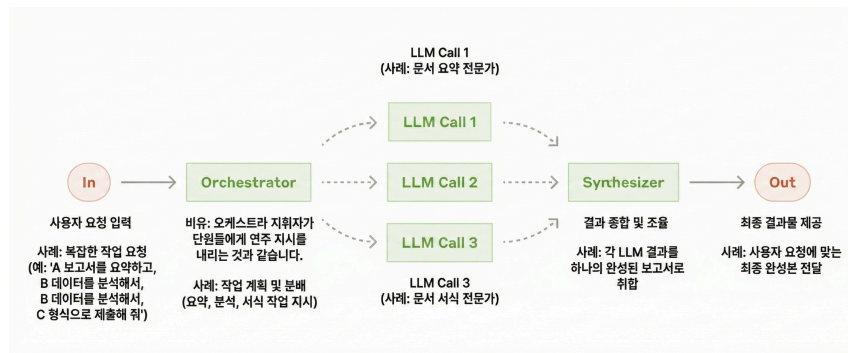
ORCHESTRATOR-WORKERS

대장 LLM이 전체 계획을 세우고 필요한 만큼 일꾼 LLM에 위임한 뒤 결과를 종합합니다. 병렬화와 달리 하위 작업이 미리 정의되지 않고 입력에 따라 동적으로 결정됩니다.

비유 지휘자가 오케스트라 단원에게 연주를 지시

조건 필요한 하위 작업을 미리 예측할 수 없을 때

예시 여러 파일에 걸친 복잡한 코드 수정, 다각도 리서치



PATTERN 05

평가자-최적화

WORKFLOW 05

피드백 루프로 완성

EVALUATOR-OPTIMIZER

생성 LLM이 답을 내놓으면 평가 LLM이 **비판하고 수정**을 요구합니다. 루프를 돌며 점차 품질을 끌어올리는 방식으로, 사람 피드백에 준하는 고품질 결과를 얻을 수 있습니다.

비유	간간한 편집자에게 원고를 교정받는 작가
조건	명확한 평가 기준 + 반복 개선이 효과적일 때
예시	문학 번역의 뉘앙스 조정, 복잡한 리서치 분석



BEYOND WORKFLOWS

자율 에이전트

AGENT

환경과 상호작용, 스스로 판단

AUTONOMOUS AGENTS

단순한 경로를 넘어, 도구 사용 결과나 코드 실행 결과(**Ground Truth**)를 보고 **다음 행동을 스스로 결정**합니다. 목적지만 알려주면 경로를 수정하며 운전하는 자율주행 자동차와 같습니다.

필수 역량	복잡한 입력 이해, 계획, 도구 사용, 오류 복구
주의	비용 ↑ · 오류 누적 위험 → 샌드박스 + 가드레일 필수
예시	GitHub 이슈를 분석해 파일 수정까지 수행하는 코딩 에이전트



ACI · AGENT-COMPUTER INTERFACE

도구 설계 3원칙

01

TOKEN BUDGET

충분한 토큰

모델이 막다른 골목에 몰리기 전에 '생각할' 공간을 충분히 제공합니다. 압축된 응답을 강요하지 마세요.

02

NATURAL FORMAT

자연스러운 형식

인터넷에서 자주 등장하는 포맷에 가깝게 설계합니다. Diff 작성보다 전체 파일 재작성이, JSON 내 코드보다 마크다운이 쉽습니다.

03

NO OVERHEAD

포맷팅 부담 제거

라인 카운팅·문자 이스케이핑 같은 오버헤드를 줄입니다. "실수하기 어렵게" 설계하는 것이 핵심 (Poka-yoke)입니다.

CLOSING

단순성 · 투명성 · ACI

KEY TAKEAWAYS

단순한 프롬프트에서 **시작**하라.

종합적 평가로 **최적화**하라.

더 단순한 방식이 **부족할 때**에만 복잡성을 추가하라.

01

Simplicity

단순성 — 불필요한 복잡성 제거

02

Transparency

투명성 — 계획 단계를 명시적으로

03

Good ACI

잘 설계된 도구 인터페이스

